

Lab Session 8: Documenting Code with Doxygen

Exercise

1. You are tasked with developing a C# program that calculates and generates a mark sheet for students. The mark sheet should include details such as student name, roll number, subject-wise scores, total marks, percentage, and grade. You want to create comprehensive documentation for your mark sheet program using Doxygen.

Code:

```
using System;
using System.Collections.Generic;

/// <summary>
/// Represents a student and provides functionality
/// to calculate total marks, percentage, and grade.
/// </summary>
public class Student
{
    /// <summary>
    /// Gets or sets the student name.
    /// </summary>
    public string Name { get; set; }

    /// <summary>
    /// Gets or sets the student roll number.
    /// </summary>
    public int RollNumber { get; set; }

    /// <summary>
    /// Stores subject-wise marks.
    /// </summary>
    public List<int> Marks { get; set; }

    /// <summary>
    /// Initializes a new instance of the Student class.
    /// </summary>
    /// <param name="name">Student name.</param>
    /// <param name="rollNumber">Student roll number.</param>
    /// <param name="marks">List of subject marks.</param>
    public Student(string name, int rollNumber, List<int> marks)
    {
        Name = name;
        RollNumber = rollNumber;
```

```
Marks = marks;
}

/// <summary>
/// Calculates the total marks obtained by the student.
/// </summary>
/// <returns>Total marks.</returns>
public int CalculateTotal()
{
    int total = 0;
    foreach (int mark in Marks)
    {
        total += mark;
    }
    return total;
}

/// <summary>
/// Calculates the percentage of marks.
/// </summary>
/// <returns>Percentage of marks.</returns>
public double CalculatePercentage()
{
    return (double)CalculateTotal() / Marks.Count;
}

/// <summary>
/// Determines the grade based on percentage.
/// </summary>
/// <returns>Grade as a string.</returns>
public string CalculateGrade()
{
    double percentage = CalculatePercentage();

    if (percentage >= 80) return "A";
    else if (percentage >= 70) return "B";
    else if (percentage >= 60) return "C";
    else if (percentage >= 50) return "D";
    else return "F";
}

/// <summary>
/// Displays the complete mark sheet.
/// </summary>
```

```

public void DisplayMarkSheet()
{
    Console.WriteLine("----- Mark Sheet -----");
    Console.WriteLine("Name: " + Name);
    Console.WriteLine("Roll Number: " + RollNumber);
    Console.WriteLine("Total Marks: " + CalculateTotal());
    Console.WriteLine("Percentage: " + CalculatePercentage());
    Console.WriteLine("Grade: " + CalculateGrade());
}
}

/// <summary>
/// Entry point of the Mark Sheet program.
/// </summary>
public class Program
{
    /// <summary>
    /// Main method that runs the application.
    /// </summary>
    /// <param name="args">Command-line arguments.</param>
    public static void Main(string[] args)
    {
        List<int> marks = new List<int> { 85, 78, 90, 88, 76 };

        Student student = new Student("John Doe", 101, marks);
        student.DisplayMarkSheet();
    }
}

```

Doxygen Output:

← → ↻ File C:/Users/android.app.CSIT/Downloads/SCD_Lab_08/html/annotated.html

Google Chrome isn't your default browser [Set as default](#)

Student Mark Sheet Program

Main Page Classes ▾

Student Mark Sheet Program
Classes ▾
Class List
Class Index
Class Members

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

- Program** Entry point of the Mark Sheet program
- Student** Represents a student and provides functionality to calculate total marks, percentage, and grade

Student Mark Sheet Program

Main Page
Classes ▾

Student Mark Sheet Program
Classes
Class List
Class Index
Class Members
All
Functions
Properties

Here is a list of all documented class members with links to the class documentation for each member:

- CalculateGrade() : **Student**
- CalculatePercentage() : **Student**
- CalculateTotal() : **Student**
- DisplayMarkSheet() : **Student**
- Main() : **Program**
- Marks : **Student**
- Name : **Student**
- RollNumber : **Student**
- Student() : **Student**

2. Create a C# program that calculates and generates a simple loan amortization schedule. This program allows the user to input loan details like the principal amount, interest rate, and loan duration, and it calculates and displays the monthly payment and an amortization schedule.

```
using System;

/// <summary>
/// Represents a loan and provides methods
/// to calculate monthly payment and amortization schedule.
/// </summary>
public class Loan
{
    /// <summary>
    /// Gets the principal loan amount.
    /// </summary>
    public double Principal { get; }

    /// <summary>
    /// Gets the annual interest rate.
    /// </summary>
    public double AnnualInterestRate { get; }
```

```
/// <summary>
/// Gets the loan duration in years.
/// </summary>
public int Years { get; }

/// <summary>
/// Initializes a new instance of the Loan class.
/// </summary>
/// <param name="principal">Loan principal amount.</param>
/// <param name="annualInterestRate">Annual interest rate (percentage).</param>
/// <param name="years">Loan duration in years.</param>
public Loan(double principal, double annualInterestRate, int years)
{
    Principal = principal;
    AnnualInterestRate = annualInterestRate;
    Years = years;
}

/// <summary>
/// Calculates the monthly payment amount.
/// </summary>
/// <returns>Monthly payment value.</returns>
public double CalculateMonthlyPayment()
{
    double monthlyRate = AnnualInterestRate / 100 / 12;
    int totalPayments = Years * 12;

    return Principal * monthlyRate /
        (1 - Math.Pow(1 + monthlyRate, -totalPayments));
}
```

```
}

/// <summary>
/// Displays the loan amortization schedule.
/// </summary>
public void DisplayAmortizationSchedule()
{
    double balance = Principal;
    double monthlyPayment = CalculateMonthlyPayment();
    double monthlyRate = AnnualInterestRate / 100 / 12;
    int totalPayments = Years * 12;

    Console.WriteLine("Month\tPayment\tInterest\tPrincipal\tBalance");

    for (int month = 1; month <= totalPayments; month++)
    {
        double interest = balance * monthlyRate;
        double principal = monthlyPayment - interest;
        balance -= principal;

        Console.WriteLine($"{month}\t{monthlyPayment:F2}\t{interest:F2}\t\t{principal:F2}\t\t{balance:F2}");
    }
}

/// <summary>
/// Entry point of the Loan Amortization program.
/// </summary>
```

```
public class Program
{
    /// <summary>
    /// Main method that runs the loan calculator.
    /// </summary>
    /// <param name="args">Command-line arguments.</param>
    public static void Main(string[] args)
    {
        Loan loan = new Loan(10000, 5, 2);

        Console.WriteLine("Monthly Payment: " + loan.CalculateMonthlyPayment());
        loan.DisplayAmortizationSchedule();
    }
}
```

Doxygen Output:

← → ↻ ⓘ File C:/Users/android.app.CSIT/Downloads/SCD_Lab_08/html/annotated.html

 Google Chrome isn't your default browser [Set as default](#)

Loan Amortization Program

Main Page [Classes](#) ▾

▾ Loan Amortization Program

▾ Classes

▸ Class List

Class Index

▸ Class Members

↔

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

 Loan	Represents a loan and provides methods to calculate monthly payment and amortization schedule
 Program	Entry point of the Loan Amortization program

Loan Amortization Program

Main Page [Classes](#) ▾

▾ Loan Amortization Program

▾ Classes

▸ Class List

Class Index

▾ Class Members

All

Functions

Properties

↔

Here is a list of all documented class members with links to the class documentation for each member:

- AnnualInterestRate : [Loan](#)
- CalculateMonthlyPayment() : [Loan](#)
- DisplayAmortizationSchedule() : [Loan](#)
- Loan() : [Loan](#)
- Main() : [Program](#)
- Principal : [Loan](#)
- Years : [Loan](#)